



5

21

2520 sp

نظري

كلية الهندسة المعلوماتية

السنة الخامسة - هندسة البرمجيات

شجرة التغطية وشبكات بتري الزمنية و RT

د. باسم قصيبة



RB Informatics; 11/11/2024

هندسة برمجيات ٣

سؤال دورة:

```
Process station;
Whiel(true)
{
    Open_Door();
    Sleep(30 seconds);
    Close_Door();
}
```

```
Process train;
While(true)
{
    Enter_Station();
    Leave_Station();
}
```

```
Process person_in_station;
Board_Train();
```

```
Process person_in_train;
Leave_Train();
```

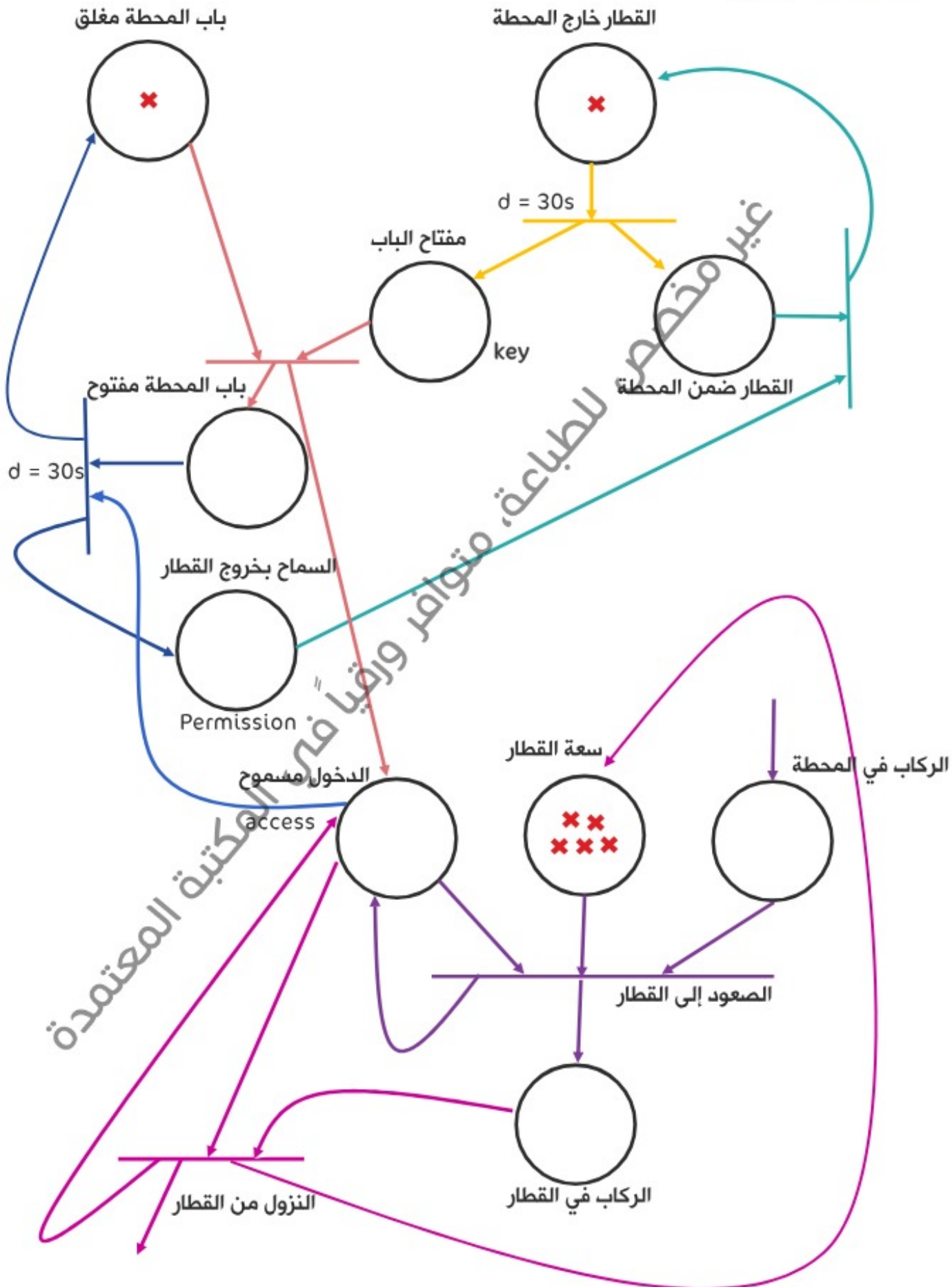
لدينا هذه الإجراءات الأربع وأفعالها (توابعها) كما يوضح الشكل السابق. نريد تنظيم (مزامنة) الأفعال بحيث:

- لا يمكن للمحطة فتح الباب إلا عندما يدخل القطار إلى المحطة.
- لا يمكن للشخص الموجود في المحطة ركوب القطار إذا كانت سعة القطار ممتلئة، وكان باب المحطة مفتوحاً.
- لا يمكن للشخص الموجود في القطار مغادرة القطار إلا إذا كان باب المحطة مفتوحاً.
- لا يمكن للقطار مغادرة المحطة إلا إذا كان المحطة مغلقاً.

1. نمذج نظام المترو (الحرج) الموصف أعلاه باستخدام شبكات بتري؟ ملاحظة: احترم شروط وقواعد رسم شبكات بتري وإلا فإن كامل السؤال خاطئ بكل طلباته.
2. اشرح فقط بحسب النموذج الذي رسمت كيف يمكنك التخطيط للتأكد (التحقق) من أنه لا يمكن لشخص الصعود والنزول من القطار إلا ضمن الشروط الآمنة؟



(1) رسم شبكة بتري:



شرح رسم شبكة بتري (غير مطلوب الشرح في الامتحان):

لدينا الحالات الأولية : القطار خارج المحطة ، باب المحطة مغلق وقد فرضت المسألة أن كل 30 ثانية يفتح باب المحطة ومن ثم يغلق.

لتبسيط رسم وفهم الشبكة المعقدة السابقة ، لنقم بتقسيم النظام لقسمين ، الأول هو القطار خارج المحطة إلى لحظة وصوله ، والثاني هو حالة الركاب داخل المحطة:

الحالة الأولى:

1. يكون القطار خارج المحطة وباب المحطة مغلق.
2. بعد 30 ثانية يصل القطار إلى المحطة ونحضر مفتاح لفتح باب المحطة والسماح بدخول الركاب (الدخول مسموح).
3. القطار ضمن المحطة يمكن أن يصبح خارج المحطة لكن مع وجود شرط متعلق بالمكان "السماح بخروج القطار".
4. وجود مفتاح باب المحطة والباب مغلق سيؤدي إلى فتح باب المحطة.
5. بعد 30 ثانية يغلق باب المحطة ويتم السماح للقطار بالخروج من المحطة.

الحالة الثانية:

1. الركاب في المحطة ولا يستطيعون ركوب القطار إلا إذا كانت سعة القطار تسمح بذلك وبشرط أن يكون الدخول مسموح (يشمل وصول القطار وفتح باب المحطة كما تم شرحه مسبقاً).
2. في حال تحققت الخطوة الأولى يصعد الركاب إلى القطار.
3. الركاب في القطار يمكنهم النزول من القطار بحال تحقق شرط الدخول مسموح (يضمن هذا الشرط أن القطار أصبح في المحطة وباب المحطة مفتوح).
4. عند نزول الركاب من المحطة يتم تحرير مكان في القطار (تزداد سعة القطار بمقدار عدد الركاب التي نزلت) والدخول مازال مسموح.

ملاحظة: رسم شبكة بتري لهذا المثال ليس بالامر السهل بل يحتاج إلى التدريب أكثر على رسم شبكات بتري حتى نستطيع نمذجة مثل هذه الأنظمة باستخدام شبكات بتري

(2) نرسم شجرة التفطية لشبكة بتري المبنية أعلاه ونراقب التالي:

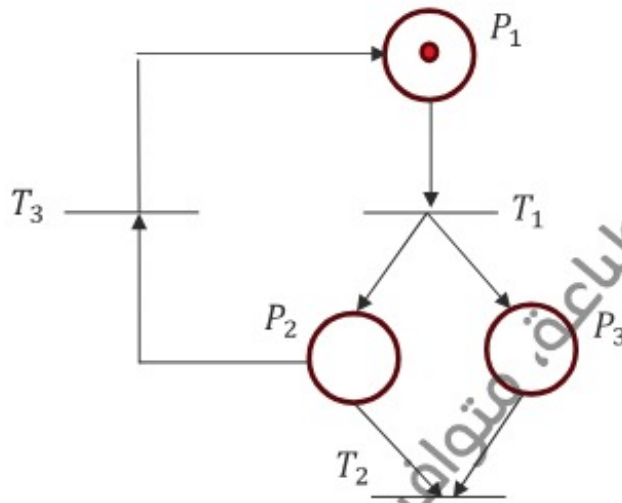
1. شعاع ترقيم فيه $access=0$ وشعاع الترقيم الذي يليه قد تغير عدد الركاب في المحطة أو القطار هذا يعني أن الركاب صعدوا أو نزلوا من القطار دون السماح لهم.
2. شعاع ترقيم فيه $permission=0$ وشعاع الترقيم الذي يليه تغير من داخل المحطة إلى خارج المحطة.. يعني أن القطار غادر دون السماح له بالمغادرة.

❖ انتهينا من السؤال، والآن نبدأ بالمحاضرة:

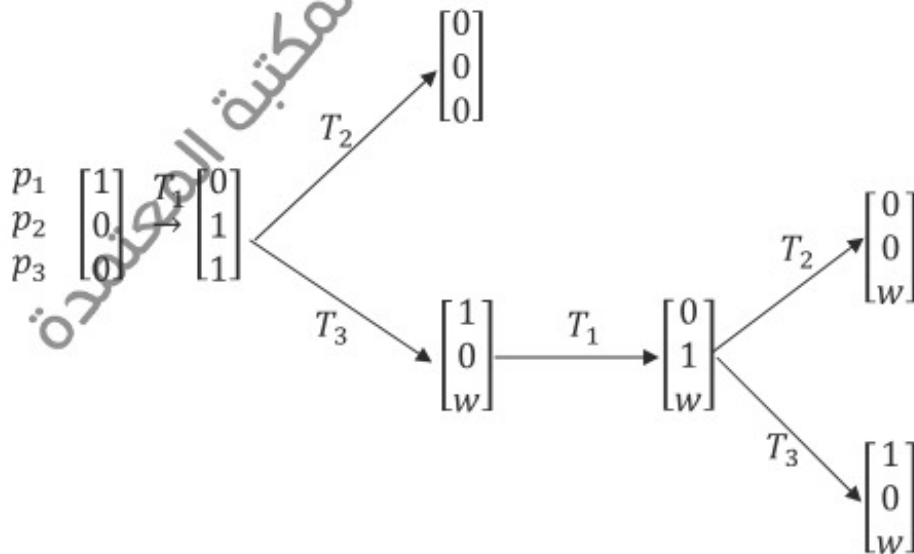
خوارزمية رسم شجرة التغطية

1. ابتداء من شعاع الترتيم الابتدائي M_0 نشير إلى كل الممرات القابلة للعبور وأشعة الترتيم الموافقة لهذه العبورات، إذا وصلنا إلى أحد أشعة الترتيم التي هي حتما أكبر من M_0 فإننا نضع w لكل مكان أكبر من الأماكن المطابقة في M_0 .
2. إذا وجد على الطريق بين M_i و M_0 شعاع M_j فإن $M_i = M_j$ لا لاحق له . وإلا نتابع من الخطوة الأولى.

مثال: ارسم شجرة التغطية للشبكة التالية:



الحل:



شرح طريقة رسم شجرة التغطية باتباع خوارزمية شجرة التغطية:

1. نضع الشعاع الابتدائي (شعاع عمودي عناصره تمثل الأماكن كما هو موضح بالشكل) حيث نعطي القيمة 1 للأماكن التي يوجد فيها علامة و 0 لغير ذلك.
2. عند عبور الممر T1 نمنح العلامة من مكان دخلها P1 ونضع علامة في أماكن الخرج P2, P3.
3. نرسم الشعاع الناتج من عبور الممر T1.
4. الشعاع الناتج من 3 يتفرع منه شعاعين.
5. في شبكات بتري من الممكن أن تنفجر شجرة التغطية أي يصبح حجمها لا يمكن دراستها ، هذه الخوارزمية تحل المشكلة وذلك بوضع w لكي لا ينفجر المكان فأعبر بذلك عن أن هذا المكان امتلئ وهو متزايد دون أن أعلم ما هو هذا التزايد لكي لا استمر بالزيادة حتى الانهائية، هنا شعاع التقييم M_i أكبر تماماً من شعاع التقييم الابتدائي لأن كل الأماكن متساوية بين الشعاعين إلا مكان أو أكثر يكون أكبر من المكان المقابل له في M_0 فنضع في هذا المكان w حسب الخطوة 1 في الخوارزمية.

للمقارنة بين شعاعين فإننا نقارن المركبات بالترتيب من الأعلى للأسفل، إذا كان أحد المركبات أكبر من الآخر يكون الشعاع صاحب هذه المركبة أكبر (نلاحظ الشعاع الناتج عند العبور من T3 أنه أكبر من الشعاع الابتدائي، فوضعها w في مكان المركبة الأكبر)، وفي حال كانت المركبتان متساويتين نقارن المركبتان التاليتان، وهكذا إلى نهاية الشعاعين، بحال تساوت جميع مركبات الشعاعين فهما متساويان.

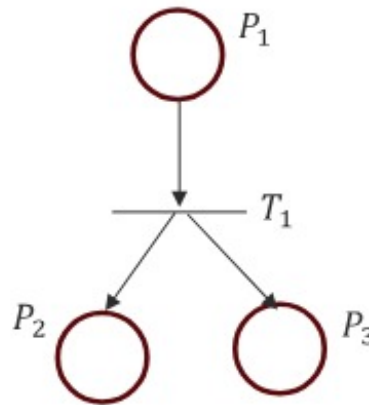
الجبر الخطي: شجرة التغطية تتعلق بالحالة الابتدائية بغض النظر عن العلامات الموجودة بالشجرة.

مفهوم تابعي ال pre وال post:

ليكن P هو المكان و T هو الممر فيكون:

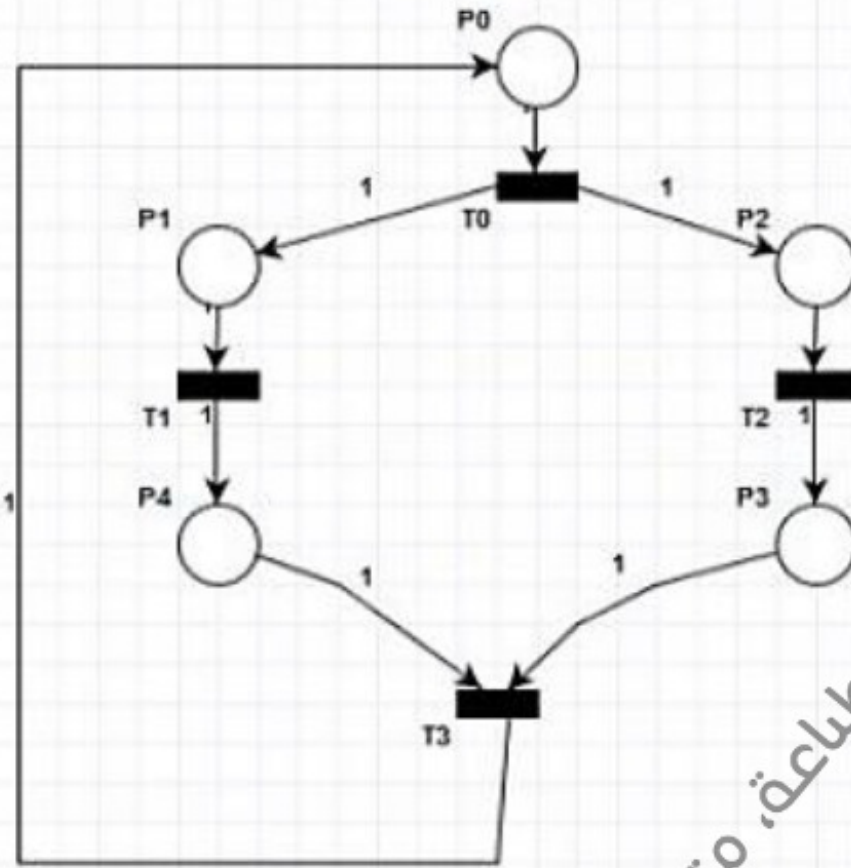
$pre(T, P)$: وزن السهم الذي يصل P ب T ويسبق T.

$post(T, P)$: وزن السهم الذي يصل T ب P ويأتي T.

مثال 1:

$$\begin{aligned}
 pre(T_1, p_1) &= 1 \\
 pre(T_1, p_2) &= 0 \\
 pre(T_1, p_3) &= 0
 \end{aligned}$$

$$\begin{aligned}
 post(T_1, p_1) &= 0 \\
 post(T_1, p_2) &= 1 \\
 post(T_1, p_3) &= 1
 \end{aligned}$$



لا يوجد أسهم تسبق T_0 ترتبط بـ p_1, p_2, p_3, p_4 لذلك $pre = 0$ في حين أن $pre(T_0, P_0) = 1$:

$$Post(T_0, P_0) = 0$$

$$Post(T_0, P_1) = 1$$

$$Post(T_0, P_2) = 1$$

$$Post(T_0, P_3) = 0$$

$$Post(T_0, P_4) = 0$$

■ وزن القوس هو عدد العلامات التي تنتقل عند كل انتقال.

■ عدم وجود قوس هو 0.

لن يرافقك في الطريق كل شيء، تختلف
الخارطة ويبقى الطريق واحداً، تختلط
الأحداث ويبقى الهدف واحداً

مصفوفة الإسقاط الأمامية (W^-) تستخدم التابع pre

	T_0	T_1	T_2	T_3
P_0	1	0	0	0
P_1	0	1	0	0
P_2	0	0	1	0
P_3	0	0	0	1
P_4	0	0	0	1

- الذي يسبق T_0 هو P_0 فأضع 1 والباقي أصفار.
- الذي يسبق T_3 هو P_4, P_3 فأضع واحد عند كل منهما والباقي أصفار وهكذا.

مصفوفة الإسقاط الخلفية (W^+) تستخدم التابع post

	T_0	T_1	T_2	T_3
P_0	0	0	0	1
P_1	1	0	0	0
P_2	1	0	0	0
P_3	0	0	1	0
P_4	0	1	0	0

- وهنا أيضا الذي يلي T_0 هو P_2, P_1 أضع 1 عند كل منهما والباقي أصفار.
- الذي يلي T_2 هو P_3 أضع 1 والباقي أصفار وهكذا.

مصفوفة الإسقاط (W)

$$W = W^+ - W^-$$

	T_0	T_1	T_2	T_3
P_0	-1	0	0	1
P_1	1	-1	0	0
P_2	1	0	-1	0
P_3	0	0	1	-1
P_4	0	1	0	-1

مصفوفة الإسقاط تعبر عن طوبولوجيا الشبكة، حيث أنه عندما أريد أن أعبر ممر وليكن T_0 (دون الرجوع للشجرة) أزيل علامة من P_0 وأضيف علامة في P_1 وعلامة في P_2 (وذلك من مصفوفة الإسقاط):

- عند عبور T_1 أزيل علامة من P_1 وأضيف علامة في P_4 .
- عند عبور T_2 أزيل علامة من P_2 وأضيف علامة في P_3 .
- عند عبور T_3 أزيل علامة من P_3 وعلامة من P_4 وأضيف علامة في P_0 .

المعادلة الأساسية للانتقال

$$M_k = M_i + W \cdot S$$

حيث:

S : هي سلسلة العبور (مصفوفة العبور).

حيث $S = T_0 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_0$ سلسلة عبور صحيحة، يجب أن تكون السلسلة valid.

M_i : شعاع التقييم الذي نبدأ العبور من عنده.

M_k : شعاع التقييم الذي سنصل إليه بعد العبور.

W : مصفوفة الإسقاط.



$$S = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

2: تواتر ورود T_0 .

1: تواتر ورود T_1 .

وهكذا..

$$M_i = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$M_k = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 & 0 & 0 & 1 \\ 1 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 1 & 0 & -1 \end{bmatrix} * \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

- معادلة الانتقال من حالة إلى حالة أخرى في شبكة بتري (شجرة التغطية) نتيجة عبور سلسلة ممرات صحيحة.
- القيم النهائية يجب أن تكون موجبة.
- M_i ليس بالضرورة أن يكون الشعاع الجذر.
- كل توزع للعلامات داخل شبكة بتري يعبر عن انتقال من انتقالات الاوتومات.

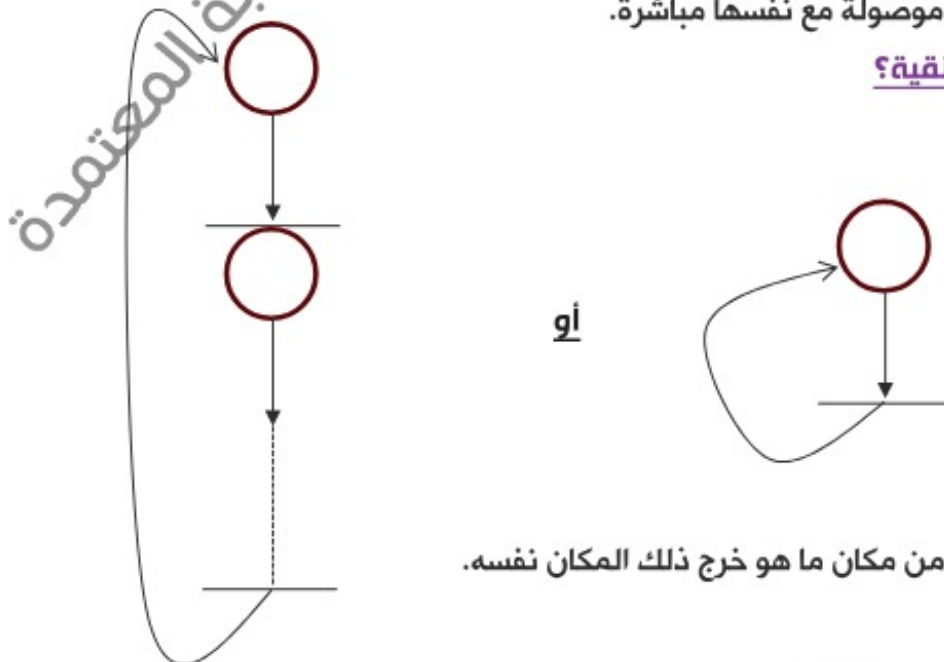
ملاحظة : يمكن تشكيل شبكة بتري من مصفوفة الإسقاط إذا كانت شبكة بتري نقية.

متى تكون الشبكة نقية؟

عندما لا يوجد فيها عقدة موصولة مع نفسها مباشرة.

متى تكون الشبكة غير نقية؟

إذا كانت من الشكل:



أي دخل الممر من مكان ما هو خرج ذلك المكان نفسه.

شبكات بترى الزمنية

في شبكات بترى العادية كنا نفترض أن العبور من مكان إلى مكان هو عبور آني (أي الحدث أو العملية تحدث مباشرة) دون أخذ الزمن بعين الاعتبار، لذا في شبكات بترى الزمنية سنضع الأزمنة على الرسم.

لكن السؤال أين سنضع الزمن (على الممر أم على المكان)؟

يوجد نوعين من شبكات بترى الزمنية:

1. شبكات بترى الزمنية بالأماكن.

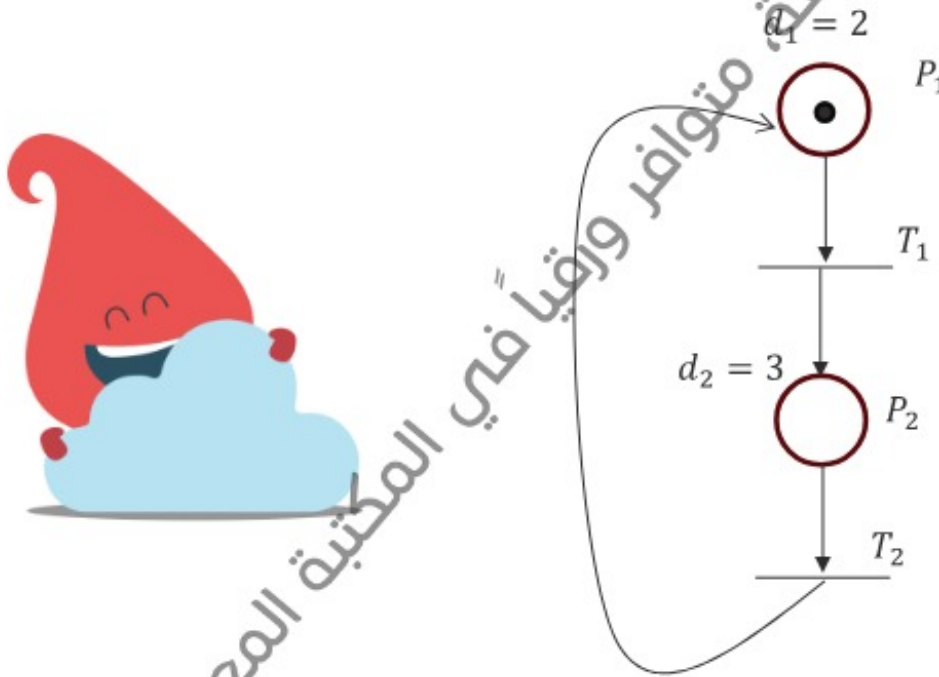
2. شبكات بترى الزمنية بالممرات.

ويمكن التحويل بين النوعين السابقين.

سنرمز إلى الزمن ب d

سنأخذ شبكات بترى الزمنية بالأماكن (أي لكل مكان يوجد زمن لتكون العلامة قابلة للاستخدام)

مثال:



لدينا علامة واحدة في المكان P_1 و $d_1 = 2$ أي نحتاج إلى وحدتين زمنييتين لتكون العلامة في المكان P_1 قابلة للاستخدام لكن بما أن المكان P_1 يمثل لحظة تشغيل النظام فلا نأخذ d_1 بعين الاعتبار (أي نعتبر $d_1 = 0$).

شجرة التغطية:

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} \xrightarrow{T_1/0} \begin{bmatrix} 0 \\ 1(3) \end{bmatrix} \xrightarrow{T_2/3} \begin{bmatrix} 1(2) \\ 0 \end{bmatrix} \xrightarrow{T_1/2} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

- نلاحظ وجود الزمن بين قوسين يعبر عن الوحدات الزمنية اللازمة لتكون العلامة قابلة للاستخدام، وكذلك لكل انتقال يوجد زمن عبور.
- نلاحظ لدينا حالتان $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ لم نقم بدمجهم وتم فصلهم بسبب اختلاف الزمن.

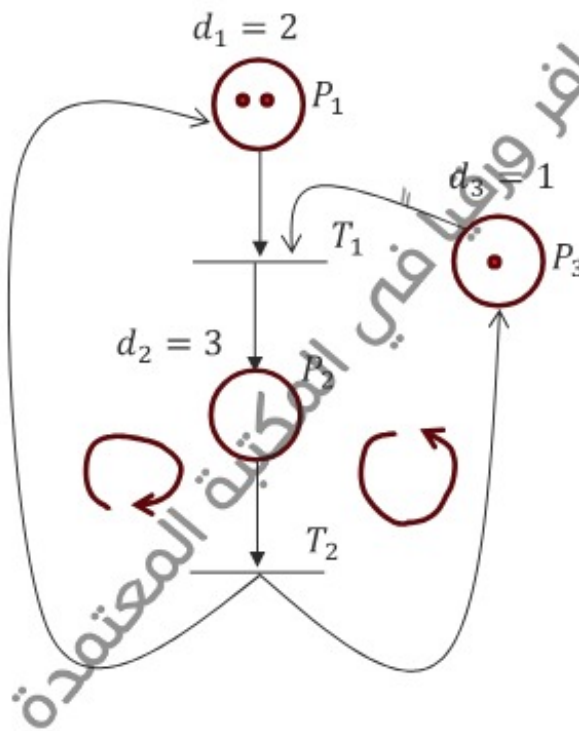
على فرض كانت الحالة الابتدائية في الشبكة بوجود علامة في كل من المكانين P1 و P2 فتكون شجرة التغطية:

$$\begin{bmatrix} 1 & (0) \\ 1 & (0) \end{bmatrix} \xrightarrow{T_1 T_2 / 0} \begin{bmatrix} 1 & (2) \\ 1 & (3) \end{bmatrix} \xrightarrow{T_1 / 2} \begin{bmatrix} 0 \\ 2 & (1,3) \end{bmatrix} \xrightarrow{T_2 / 1} \begin{bmatrix} 1 & (2) \\ 1 & (2) \end{bmatrix}$$

$T_1 T_2 / 2$

نلاحظ اختلاف شجرة التغطية لنفس شبكة بتري عند اختلاف العلامات في الأماكن، فلو كان هناك ثلاث علامات في أماكن معينة سينتج لدينا شجرة تغطية جديدة وهكذا...

مثال: في المثال السابق قمنا بحل المسألة بالسرعة القصوى (أفضل زمن) وفي هذا المثال سنرى الحالة الأسوأ للزمن.



- الأسهم الحمراء عبارة عن أي process لدينا 2 process.

■ نحسب تردد ال process الأولى (اليسرى):

$$d_1 F_1 + d_2 F_2 = 2$$

مجموع عدد العلامات

في المكانين P1 و P2

$$F_1 = F_2 = F'$$

$$(d_1 + d_2) F' = 2$$

$$5F' = 2 \rightarrow F' = \frac{2}{5}$$

■ نحسب تردد ال process الثانية:

$$d_3 F_1 + d_2 F_2 = 1$$

$$F_1 = F_2 = F''$$

$$(d_3 + d_2) F'' = 1 \rightarrow F'' = \frac{1}{4}$$

$$\max F = \min(F', F'') = \frac{1}{4}$$

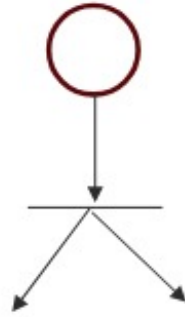
■ ال F هو التردد (تردد ال process)

■ F_1 تردد مرور العلامات للممر الأول T_1

■ F_2 تردد مرور العلامات للممر الأول T_2

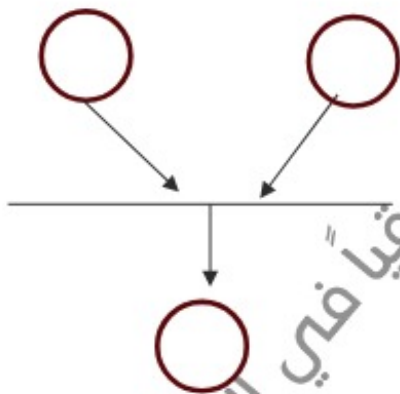
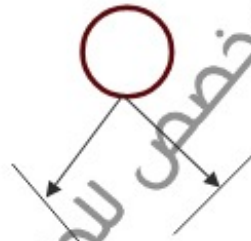
Sequential





fork

Optional (if -else)



join

وهناك أيضا نموذج خامس وكل نموذج له طريقة ليتحول إلى كود برمجي.

شبكات بتري الزمنية تفيد في قياس أو التأكد من أن نظام ما هو نظام زمن حقيقي.

نظم الزمن الحقيقي

- نظام الزمن الحقيقي: هو النظام حيث أن صحته لا تعتمد فقط على صحة نتائج حساباته المنطقية وإنما على الزمن الذي تم إنجاز هذه العمليات فيه.
- في نظم الزمن الحقيقي: التوقيت يرتبط بقوة بوظيفة النظم وأدائها وموثوقيتها.

مثال 1:

(توقيت الأحداث مهم جداً في نظم الزمن الحقيقي) كيس الهواء في السيارة يجب أن يفتح باللحظة المناسبة عند حدوث الحادث... ليس قبل وليس بعد لحظة معينة.

مثال 2 نظام للجراحة عن بعد:

يستخدم الجراح الأدوات بحيث يقوم روبوت بمحاكاة أفعال الجراح على جسد المريض. يجب الأخذ بعين الاعتبار عطالة محركات الروبوت بدقة حيث أن الأمر يصدر باللمحة t ولكنه بحاجة إلى (Δt) للتنفيذ (عطالة المحركات). وبالتالي يجب الأخذ بعين الاعتبار إصدار الأمر قبل $(-\Delta t)$ من الزمن المطلوب، وبشكل مشابه كما في المثال 3.

مثال 3: نظام فرملة السيارة.

- مما سبق نلاحظ أن عمر المعطيات مهم في اتخاذ القرار، فالتصرف يصبح خاطئاً إذا ما تم الاعتماد على معطيات لم تغد صحيحة.
- نلاحظ أن نظام الزمن الحقيقي متزامن مع البيئة المتفاعلة معه.

متطلبات نظم الزمن الحقيقي:

يجب أن يملك سرعة معالجة كافية وسرعة اتصال كافية بين أجزائه الموزعة، وكما يجب أن يعالج مشاكل تشارك الموارد بين فعالياته والتزامن بينها.

أنصاف نظم الزمن الحقيقي**1. Hard real-time systems**

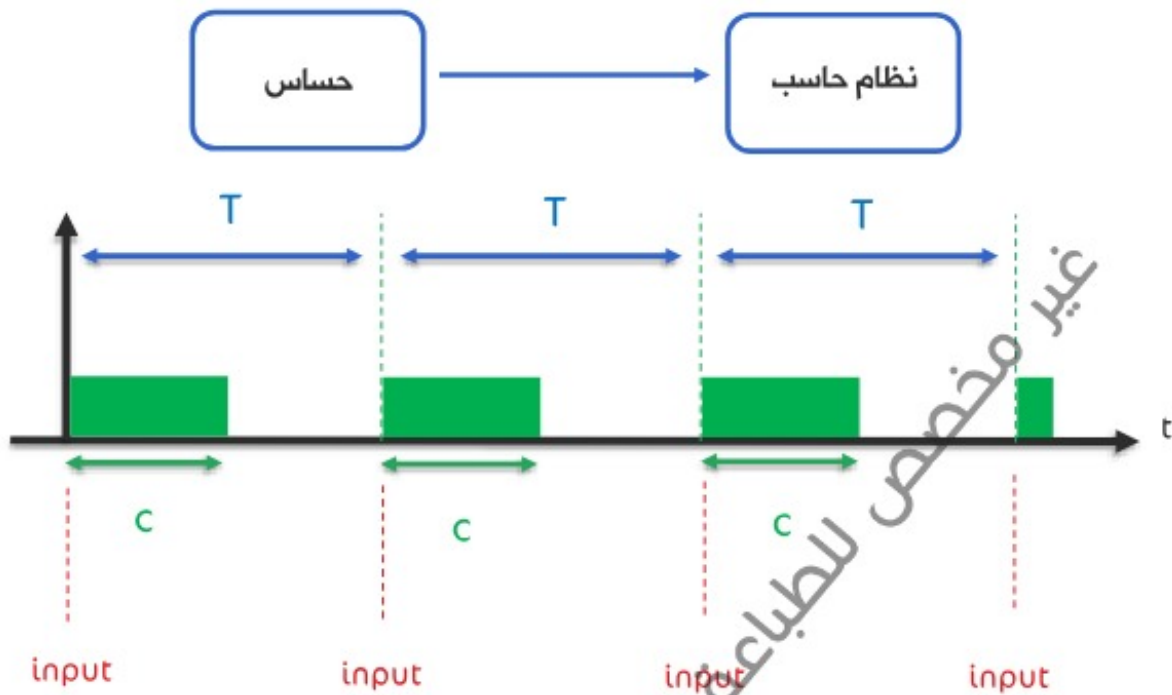
إذا تم الحدث بعد انقضاء الموعد المحدد له (deadline) فإننا نعتبر بأن النظام قد فشل. تعتبر معظم هذه الأنظمة أنظمة تحكم (control systems)، ولكن ليس كل أنظمة التحكم هي أنظمة زمن حقيقي.

2. Soft real-time systems

حتى لو جاء الحدث بعد انقضاء الموعد المحدد له (deadline) فقد لا نعتبر بأن النظام فشل وإنما يمكننا اعتبار أن نتائجه مازالت مقبولة. مثال: أنظمة نقل الصوت والصورة عبر الانترنت حيث يمكننا التغاضي عن تأخر عدد من إطارات الصورة أو الإشارات الصوتية.

ملاحظة هامة جداً:

ليس من الضروري أن يكون نظام الزمن الحقيقي سريعاً وإنما يجب أن يتم تنفيذ مهامه بحيث يحقق متطلبات التوقيت والتزامن، أي أنه من أجل كل مهمة أو حدث يجب أن يتم إتمامها عند الزمن T بحيث أن $T_{min} \leq T \leq T_{max}$



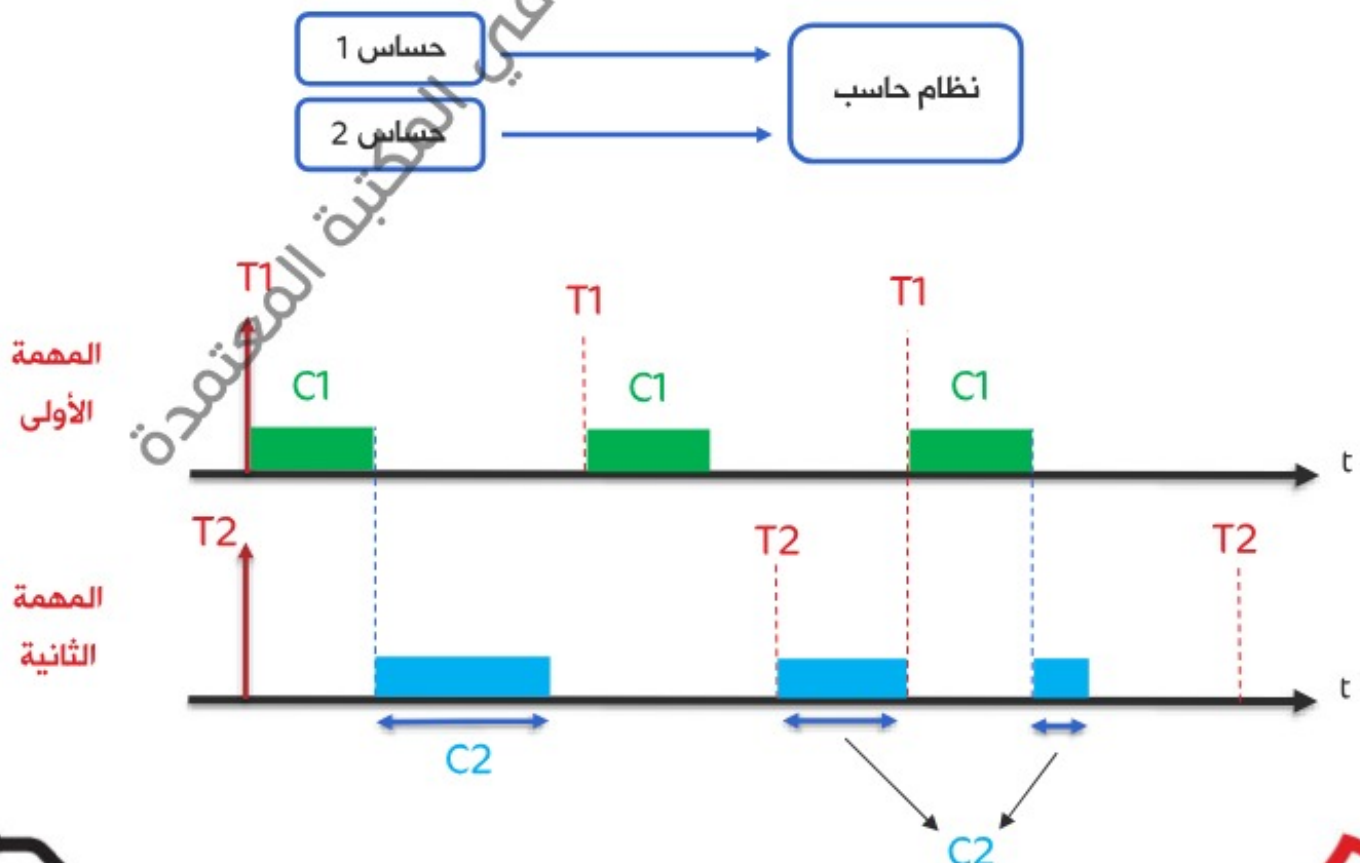
في هذا النظام البسيط نقول أن الـ D (deadline)

$$C \leq D \leq T$$

وفي حال وجود عطالة (عطالة محركات - عطالة حساسات) ولنفرض (Δt) فإن:

$$C \leq D \leq T - \Delta t$$

لنفرض وجود حساسين مختلفين:



سنقول بأن النظام نظام زمن حقيقي إذا كل مهامه حققت موعدها الأول (أو لاقت موعدها الأول).
يعني كل مهمة أنجزت موعدها الأول ضمن المحددات الزمنية المفروضة عليها $(T - \Delta t)$.

لماذا الموعد الأول تحديداً؟

لأنه الأكثر قساوة حيث تبدأ كل المهام معاً وهي أقسى حالة يمكن أن تمر في حياة النظام، فإذا استطاع النظام تجاوزها فإنه يستطيع تجاوز جميع الأوضاع الأخرى لأنها أقل قساوة وشدة

الجدولة مع أولوية:

إن التنفيذ الذي يعتمد على الأولوية (priority-based-execution) ينص على:

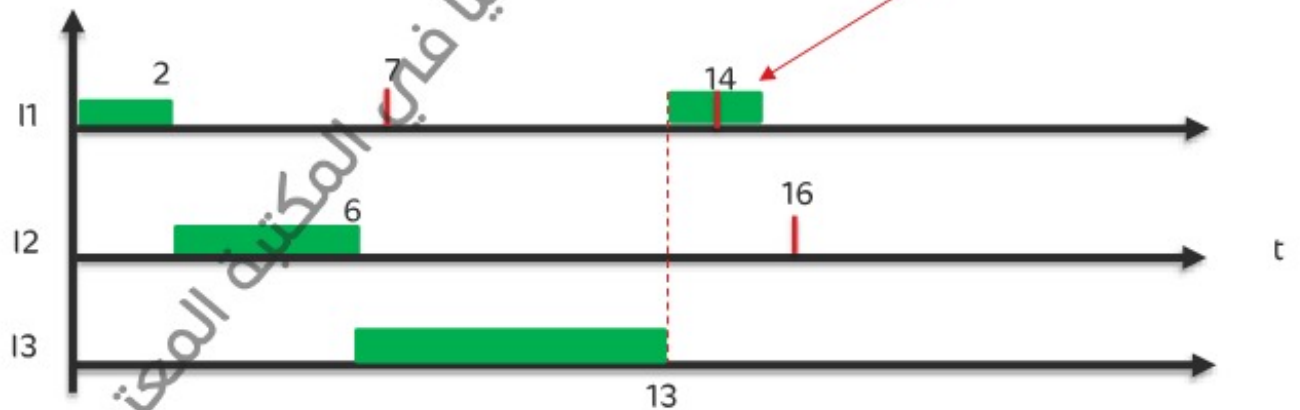
عندما يصبح المعالج عاطل عن العمل فإن المهمة ذات الأولوية الأعلى يتم اختيارها للتنفيذ وتستمر بالتنفيذ حتى النهاية.

مثال:

لدينا المهام التالية:

	Priority	Period	Computing time
I1	1	7	2
I2	2	16	4
I3	3	31	7

يوجد مشكلة هنا: يجب ان ينتهي التنفيذ قبل 14



وبالتالي النظام ليس بنظام زمن حقيقي (لأن طريقة جدولة المهام خاطئة)

السبب في أن النظام السابق ليس بنظام حقيقي هو في الدورة الثانية في اللحظة 14 يجب أن تكون المهمة I1 حققت موعدها الأول ونفذت لكن في الحقيقة انتهت المهمة I1 موعدها في اللحظة 15 وبالتالي النظام ليس بنظام زمن حقيقي.

التنفيذ بطريقة pre-emptive priority-based execution

إن التنفيذ بطريقة pre-emptive priority-based execution ينص على:
عندما يصبح المعالج عاطل عن العمل فإن المهمة ذات الأولوية الأعلى يتم اختيارها ولكن عند أي زمن (لحظة) تصبح مهمة ذات أولوية أعلى جاهزة للعمل فإنه يتم pre-emptive المهمة التي في المعالج كي يحل محلها المهمة ذات الأولوية الأعلى الجاهزة للعمل في النظام.
إذا بأي لحظة زمنية إما المعالج عاطل عن العمل أو ينفذ مهمة جاهزة (المهمة ذات الأولوية الأعلى).

لنرى سيناريو تنفيذ المهمات المذكورة في الجدول السابق بطريقة pre-emptive:



نلاحظ أنه باستخدام ال pre-emption مع الجدولة بالأولوية تم تجاوز المشكلة السابقة والنظام يحقق محدداته الزمنية. وبالتالي النظام أصبح نظام زمن حقيقي.

النموذج الحسابي البسيط للتحقق من نظم الزمن الحقيقي

المهام مستقلة تماماً والجدولة هي RM

لدينا مجموعة من الشروط للتحقق من أن النظام هو نظام زمن حقيقي:

■ الشرط الأول: شرط لازم غير كاف:

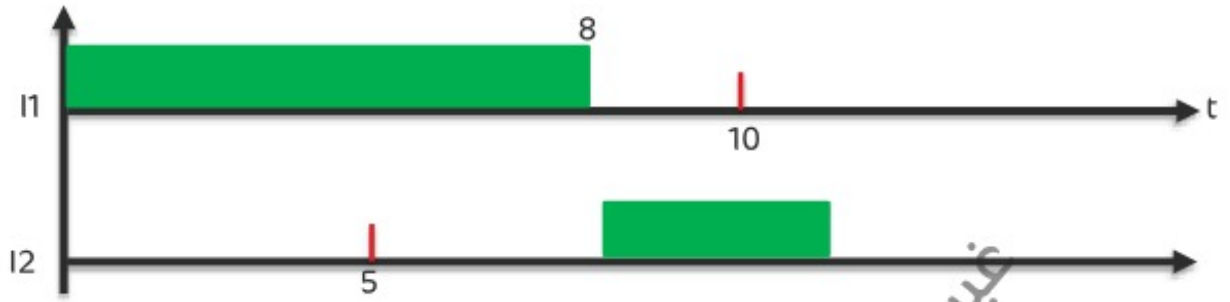
$$\forall i \ C_i < T_i$$

أي أنه يجب أن يكون زمن حساب المهمة أصغر من دورها، ولكنه غير كافٍ بأن يكون النظام هو نظام زمن حقيقي.

مثال:

	Priority	Period	Computing time
I1	1	10	8
I2	2	5	3

المثال السابق يعني بأن هذا الشرط ضروري ولكنه غير كافٍ.



لم تستطع المهمة I2 تحقيق موعدها الأول وبالتالي النظام ليس بنظام زمن حقيقي.

الشرط الثاني: شرط لازم غير كافٍ:

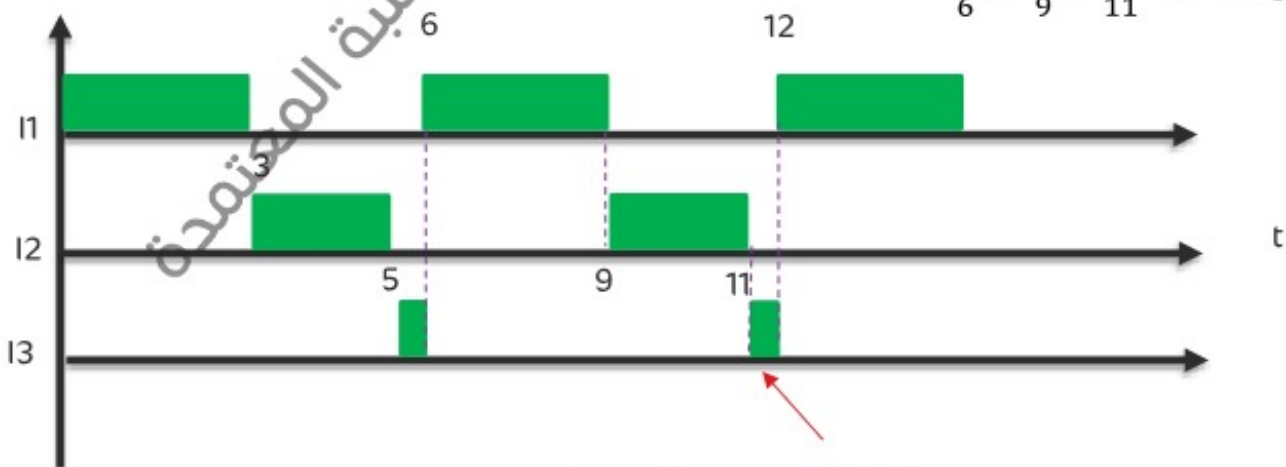
$$\sum_{i=1}^n \left(\frac{C_i}{T_i} \right) \leq 1$$

أي أن الحمولة الوسطية للنظام أصغر أو تساوي 1.

لنتأكد من أن هذا الشرط ضروري لكنه غير كافٍ نأخذ المثال التالي:

	Priority	Period	Computing time
I1	1	6	3
I2	2	9	2
I3	3	11	2

$$\frac{3}{6} + \frac{2}{9} + \frac{2}{11} \leq 1 \quad \text{نلاحظ أن:}$$



يوجد مشكلة هنا أن المهمة I3 لم تحقق موعدها الأول

وبالتالي النظام ليس بنظام زمن حقيقي.

الشرط الثالث: شرط لازم غير كافٍ:

$$\forall i \sum_{j=1}^{i-1} \left(\frac{T_i}{T_j} \times C_j \right) \leq T_i - C_i$$

وهو يعني بأنه يجب معالجة كل المقاطعات

من المهام ذات الأولوية الأعلى من i ضمن الزمن $T_i - C_i$ الحر عند المستوى i .

كل المهام ذات الأولوية الأعلى من i

المشكلة أن $\frac{T_i}{T_j}$ ليس بحاصل قسمة صحيح وهو يؤدي لمشاكل..

الشرط الكاف غير اللازم:

$$n \left(2^{\frac{1}{n}} - 1 \right) \geq \sum_{i=1}^n \left(\frac{C_i}{T_i} \right)$$

عدد المهام في النظام

ماذا يعني بأنه شرط كافٍ غير لازم؟.. أي أنه إذا تحقق فالنظام حتماً

هو نظام زمن حقيقي ولكن إذا لم يتحقق قد يكون النظام هو نظام زمن حقيقي.

الشرط الأكيد:

$$R_i^{n+1} = C_i + \sum_{j=1}^{i-1} \left\lfloor \frac{R_i^n}{T_j} \right\rfloor \times C_j$$

مقدار الزيادة على زمن تنفيذ المهمة i بسبب قطع المهام ذات الأولوية الأعلى للمهمة i (خلال زمن استجابتها الحقيقي)

زمن الاستجابة الحرج للمهمة i

زمن الحساب الاصلي للمهمة i

حتى نفهم كيف يمكن حساب أزمنة الحساب الحرج للمهام في نظام نأخذ المثال التالي:

مثال:

	Priority	Period	Computing time
I1	1	10	1
I2	2	12	2
I3	3	30	8
I4	4	600	20

أولاً: نحسب الحمولة الوسطية للنظام.

$$\frac{1}{10} + \frac{2}{12} + \frac{8}{30} + \frac{20}{600} \leq 1$$

إذا تحقق هذا الشرط نتابع وإلا فالنظام ليس بنظام زمن حقيقي.

ثانياً: نحسب زمن الاستجابة الحرج لكل مهام النظام ويجب أن يكون

$$R_i \leq T_i - D_i$$

لنحسب زمن الاستجابة الحرج للمهمة 4:

$$R_4 [0,20[= 2 \times 1 + 2 \times 2 + 1 \times 8 = 14$$

عدد مرات قطع المهمة 1 للمهمة 4

زمن حساب
المهمة 1

عدد مرات
قطع المهمة
2 للمهمة 4

زمن حساب
المهمة 2

عدد مرات
قطع المهمة
3 للمهمة 4

زمن حساب
المهمة 3

$$R_4 [20,34[= 2 \times 1 + 1 \times 2 + 1 \times 8 = 12$$

$$R_4 [34,46[= 1 \times 1 + 1 \times 2 = 3$$

$$R_4 [46,49[= 1 \times 2 = 2$$

$$R_4 [49,51[= 1 \times 1 = 1$$

$$R_4 [51,52[= \emptyset$$

إذا زمن الاستجابة الحرج للمهمة 4:

$$R_4 = 52$$

وبما أنه أصغر من 600 فإن R_4 تحقق موعدها الأول.

$$R_1 = 1 \leq 10$$

إن زمن الاستجابة الحرج للمهمة 1 هو نفسه زمن حسابها وذلك لأنها الأعلى أولوية وبالتالي لا أحد يقطعها ويأخذ المعالج منها.

زمن الاستجابة الحرج للمهمة 2:

$$R_2 [0,2[= 1 \times 1 = 1$$

$$R_2 [2,3[= \emptyset$$

$$R_2 = 3 \leq 12$$

إن زمن الاستجابة الحرج للمهمة 3:

$$R_3 [0,8[= 1 \times 1 + 1 \times 2 = 3$$

$$R_3 [8,11[= 1 \times 1 = 1$$

$$R_3 [11,12[= \emptyset$$

$$R_3 = 12 \leq 30$$

وبما أن كل مهام من النظام تحقق موعدها الأول فإن النظام هو نظام زمن حقيقي.



لا تتأل الكمالات إلا على جسر من التعب، والنعيم لا يدرك بالنعيم